Link:
https://colab.research.google.com/drive/1s06sEeMPooGlg0syKnoly7g3DZ4aR4-P?usp=sharing

Introduction

When reviewing the original SSM/RNN convolution homework code, I noticed that although the mathematical ideas were elegant, the surrounding implementation suffered from several software-engineering issues. Many functions lacked docstrings, interactive widgets were mixed with computation-heavy benchmarking logic, and some functions were accidentally redefined—making the notebook harder to maintain, reuse, or expand.

To address this, I updated the notebook to improve clarity, structure, and coding style while preserving all pedagogical TODOs. The refactored version introduces clearer organization, stronger documentation, and a separation of computation from visualization. These changes better align the assignment with standard Python and ML engineering practices while still keeping the learning goals intact.

Improvements

For explanatory paragraphs, I used ChatGPT with a structured refinement prompt (the same one used in the example). This ensured that descriptive text stayed:

- technically accurate
- concise and well-organized
- consistent with deep learning terminology
- aligned with the original meaning

These refinements improved readability without altering any mathematical content.

The original notebook performed heavy benchmark computations at import time and mixed measurement logic with interactive plotting. I split this code into two clearly separated components:

`diag_run_benchmark()` — performs only the computation

`create_interactive_diag_benchmarks()` — builds the interactive slider+plot UI

This follows the single-responsibility principle and matches modern notebook organization.

Main changes include:

- Added descriptive docstrings to all functions
    - Enhances readability, documentation, and instructional clarity.
    - Reference: PEP 257 – Docstring Conventions
- Added type annotations (e.g., List[float], Dict[int, List[int]])

        - Improves clarity, supports static analysis, and reduces ambiguity in student-facing code.
        - Reference: PEP 484 – Type Hints
- Eliminated duplicate function names
    - The original notebook defined the function interactive_benchmark twice, causing the first definition to be silently overwritten.
    - I renamed the diagonal version to interactive_diag_benchmark and wrapped both inside clearly named creator functions.
    - Avoid name collisions and maintain module clarity
    - Reference: PEP 8 — Naming Conventions
- Separated computation from visualization
    - Originally, benchmarks ran automatically and the widget logic was intermixed with numerical code.
    - Now:
        - **`diag_run_benchmark()`** only computes & returns results
        - **`create_interactive_diag_benchmarks()`** only creates UI

Now we call:

```
diag_T_cache, diag_unrolled_cache, diag_conv_cache =
diag_run_benchmark()

create_interactive_diag_benchmarks(diag_T_cache, diag_unrolled_cache,
diag_conv_cache)
```

This reduces side effects and improves modularity.

Reference: Google Python Style Guide — Functions Should Have Minimal Side Effects

- Improved code organization and readability
    - Grouped related functions
    - Reordered imports
    - Used more descriptive variable names
    - Ensured logical flow from utility → computation → visualization
    - This adheres to the broader software-engineering principle of “clean, modular design.”